

Manual Técnico

para Desenvolvedores

Configuração · Escalabilidade · Operacionalização

STACK ATUAL (PROTÓTIPO)

Frontend:	React 18 · JSX · CSS-in-JS — 6.954 linhas em 2 arquivos JSX
Motor:	JavaScript client-side · OU + GARCH + Kyle · 674 linhas
Auth:	Mock USERS_DB plaintext · SHA-256 admin · WebAuthn
IA:	Anthropic claude-sonnet-4 direto do browser (chave exposta)
Estado:	115× useState · sem persistência entre sessões
Pagamentos:	Mock UX simulado com setTimeout

STACK ALVO (PRODUÇÃO)

Backend:	Node.js 22 · Fastify · PostgreSQL 16 · Redis 7
Realtime:	WebSockets Socket.io · Redis pub/sub · single-leader motor
Auth:	JWT RS256 · OAuth2 · WebAuthn · bcrypt · rate limiting
Pagamentos:	Mercado Pago SDK · webhooks HMAC · assinaturas recorrentes
IA proxy:	API Gateway → Anthropic (chave nunca no frontend)
Infra:	Docker · Render/Railway · Cloudflare CDN · Sentry

SUMÁRIO

1. Visão Geral da Arquitetura

- 1.1 Estado atual
- 1.2 Fluxo de dados
- 1.3 Gaps críticos

2. Motor de Mercado

- 2.1 Camadas e algoritmos
- 2.2 Parâmetros (MP{})
- 2.3 Constantes configuráveis
- 2.4 Migração para servidor

3. Backend — Roadmap

- 3.1 Stack recomendada
- 3.2 Schema PostgreSQL
- 3.3 Endpoints REST
- 3.4 WebSockets

4. Autenticação e Segurança

- 4.1 Sistema atual (mock)
- 4.2 JWT RS256
- 4.3 WebAuthn
- 4.4 Proxy Anthropic
- 4.5 Rate limiting

5. Integrações Externas

- 5.1 Anthropic API
- 5.2 Mercado Pago
- 5.3 News Server

6. Sistema de Permissões (Admin)

- 6.1 Matriz ROLE_TABS
- 6.2 Hierarquia de roles
- 6.3 ROLE_USER_ACTIONS

7. Configurações do Protótipo

- 7.1 Constantes do motor
- 7.2 IMPACT_MATRIX
- 7.3 SESSIONS

- 7.4 AGENT_PROFILES

8. Deploy e Infraestrutura

- 8.1 Variáveis de ambiente
- 8.2 Docker Compose
- 8.3 CI/CD

9. Escalabilidade e Performance

- 9.1 Projeção de carga
- 9.2 Cache Redis
- 9.3 Motor distribuído

10. Conformidade e LGPD

Visão Geral da Arquitetura

1.1 Estado Atual

O FootStock é atualmente uma aplicação **100% client-side** em React/JSX. Todo o estado (saldo, carteira, ordens, motor de preços) vive na memória do browser e é perdido ao recarregar a página. A estrutura atual é composta por dois artefatos:

Arquivo	Linhas	Descrição
footstock-gold.jsx	6.954	App principal: 45 funções, 115× useState, motor de mercado (674 linhas), 11 telas, 40 ativos
footstock-admin-gold.jsx	3.008	Painel admin: 8 módulos, sistema de permissões RBAC, WebAuthn, 5 roles

1.2 Fluxo de Dados Client-Side

O fluxo atual funciona assim:

- **1. App carrega** → estados inicializados em memória com valores hardcoded (USERS_DB, CLUBS[], preços IPO calculados)
- **2. Motor inicia** → setInterval(2s) dispara ouStep() para cada ativo: Ornstein-Uhlenbeck + GARCH + Kyle's Lambda
- **3. Usuário opera** → registerOrder() aplica Kyle, atualiza imb[], debita saldo e atualiza wallet via useState()
- **4. Notícia dispara** → newsAbsorption[] distribui impacto no FV ao longo de 40 ticks (~80s)
- **5. Reload da página** → tudo perdido. Sem persistência de nenhum estado.

1.3 Gaps Críticos para Produção

■ CRÍTICO	Chave API exposta — Anthropic API chamada diretamente do browser. Qualquer usuário pode inspecionar e usar a chave. Custo ilimitado em caso de abuso.
■ CRÍTICO	Sem persistência — 115 estados voláteis: wallet, orders, balance, txLog, shorts. Perdem-se a cada reload. Sem banco de dados.
■ CRÍTICO	Auth plaintext — USERS_DB com senhas em texto puro no bundle JS público. Admin usa SHA-256 fixo hardcoded.
■ MÉDIO	Motor multi-usuário — Cada browser roda seu próprio motor → preços divergem. Sem fonte única de verdade.
■ MÉDIO	Sem WebSockets — Atualizações de preço não propagam. Cada usuário vê sua própria simulação independente.
■ BAIXO	Responsividade — Container fixo 390×844px sem media queries. Não funciona em desktop/tablet.

02

Motor de Mercado

2.1 Camadas e Algoritmos — useMarket() hook (674 linhas)

O motor implementa 6 camadas de realismo financeiro. Cada tick (~2s) executa o pipeline completo para os 40 ativos:

Camada 1 — Ornstein-Uhlenbeck (Mean Reversion)

$dP = \theta * (FV - P) * dt + \sigma * dW$. Preço converge ao Fair Value. Theta e sigma calibrados por ativo em `MP{}`.

Camada 2 — GARCH Lite

$\text{Variance}_t = 0.000002 + 0.12 * r^2 + 0.85 * \text{Var}_{t-1}$. Vol sobe após retornos extremos. Cap de 1.8× para evitar spikes.

Camada 3 — Order Flow Imbalance (OFI)

Desequilíbrio buy/sell acumulado. Decay: ~0.91 para Série A grande, ~0.97 para B ilíquida. Influencia direção.

Camada 4 — Kyle's Lambda

Market impact = $\lambda * \text{qty} / \text{float}$. Lambda proporcional a $1/\text{float}$. `supplyScaling()` amplifica quando `supply < 5%`.

Camada 5 — Impact Matrix (Notícias)

Fluxo sintético em `PRESSURE_SPREAD_TICKS=10` ticks. FV ajustado via `ABSORPTION_TICKS=40`. Cap spot: $\pm 2.5\%$.

Camada 6 — Circuit Breaker

Variação $\geq 8\%$ no dia → suspensão de 5min. FV clampado ao spot para evitar salto na liberação.

2.2 Parâmetros de Microestrutura (MP{ })

Parâmetro	Tipo	Descrição	URU3	CAV4 (B)
<code>vol</code>	float	Volatilidade base/tick (sigma OU)	0.0018	0.0040
<code>theta</code>	float	Velocidade de mean-reversion	0.12	0.25
<code>float</code>	int	Cotas em circulação (liquidez)	6.800.000	350.000
<code>spread</code>	float	Spread base bid/ask (% preço)	0.003	0.012

2.3 Constantes Configuráveis do Motor

JAVASCRIPT

```
// useMarket() – constantes críticas
const VELOCITY_CAP          = 0.0035; // máx variação/tick (0.35%)
const DAILY_VOL_TARGET      = 0.025;  // meta vol diária (2.5%)
const ABSORPTION_TICKS      = 40;      // ticks p/ absorver notícia (~80s)
const FUNDAMENTAL_REVERSION_RATE = 0.003; // convergência FV/tick
const PRESSURE_SPREAD_TICKS = 10;      // ticks p/ distribuir ordens sintéticas
const HISTORY_LEN           = 15;      // buffer de preços p/ momentum

// Circuit Breaker: 8% intraday → freeze 5min
if (dailyPct >= 0.08) { freeze(ticker); setTimeout(unfreeze, 300_000); }

// Ranges de IPO por divisão (escala log)
RANGES = { A: [8, 40], B: [3, 12] } // FS$ min/max por série
```

2.4 Migração do Motor para Servidor

■ Requisito Crítico de Consistência

O motor DEVE rodar em um único processo servidor para garantir que todos os usuários vejam os mesmos preços. Use Redis pub/sub para broadcast e leader election via Redlock.

JAVASCRIPT

```
// backend/market-engine.js – esqueleto
const io = require('socket.io')(server);
const redis = require('ioredis')(process.env.REDIS_URL);

// Estado compartilhado (Redis, não memória)
let prices = {}, fv = {}, imb = {};

async function tick() {
  const session = getMarketSession();
  if (session.id === 'closed') return;
  for (const club of CLUBS) {
    prices[club.ticker] = ouStep(prices[club.ticker], fv[club.ticker], MP[club.ticker]);
    applyGarch(club.ticker);
    applyCircuitBreaker(club.ticker);
    await redis.hset("prices", club.ticker, prices[club.ticker]);
  }
  io.emit('prices', prices); // broadcast todos os clientes
  io.emit('book', buildBooks(prices));
}
setInterval(tick, 2000);

// React client – substituir useState por socket
socket.on('prices', data => setPrices(data));
```

Backend — Roadmap de Implementação

3.1 Stack Recomendada

Camada	Tecnologia	Justificativa
API Server	Node.js 22 + Fastify	Baixa latência, schema validation, plugins ecosystem
Banco Principal	PostgreSQL 16	JSONB, transações ACID, índices parciais em orders.status
Cache / Realtime	Redis 7	Estado do motor, sessões, rate limiting, pub/sub
WebSockets	Socket.io 4 + Redis adapter	Broadcast de preços em < 50ms para N instâncias
Auth	JWT RS256 + Passport.js	Tokens assimétricos, refresh rotation, OAuth2
Pagamentos	Mercado Pago Node SDK	Webhooks de assinatura, Pix, cartão, já mapeado na UX
IA Proxy	Fastify route → Anthropic	Chave nunca exposta; rate limit por user_id
News Server	footstock-news-server (Node)	RSS + Claude Haiku → classificação automática
Deploy	Docker + Render/Railway	PaaS gerenciado, auto-scale, zero-downtime
Monitoramento	Sentry + Datadog	Error tracking, APM, alertas circuit breaker

3.2 Schema de Banco de Dados (PostgreSQL)

SQL

```
-- Tabelas principais
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,    -- bcrypt rounds=12
  plan        TEXT DEFAULT 'Jogador',
  balance     NUMERIC(12,2) DEFAULT 2000,
  kyc_status  TEXT DEFAULT 'pending',
  role        TEXT DEFAULT 'user',
  suspended   BOOLEAN DEFAULT false,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE wallet (
  user_id     UUID REFERENCES users(id) ON DELETE CASCADE,
  ticker      CHAR(4) NOT NULL,
  qty         INTEGER NOT NULL CHECK (qty > 0),
  avg_price   NUMERIC(10,4) NOT NULL,
  PRIMARY KEY (user_id, ticker)
);

CREATE TABLE orders (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id),
  type        TEXT NOT NULL,    -- market|limit|oco|short|scheduled
  side        TEXT NOT NULL,    -- buy|sell
  ticker      CHAR(4) NOT NULL, qty INTEGER NOT NULL,
  price       NUMERIC(10,4),    -- NULL para market
  sl_price    NUMERIC(10,4),    -- OCO stop loss
  tp_price    NUMERIC(10,4),    -- OCO take profit
  status      TEXT DEFAULT 'pending',
  scheduled_at TIMESTAMPTZ, executed_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Índices críticos
CREATE INDEX idx_orders_user_status  ON orders(user_id, status);
CREATE INDEX idx_orders_ticker_pending ON orders(ticker) WHERE status='pending';
CREATE INDEX idx_tx_user_date        ON transactions(user_id, created_at DESC);
```

3.3 Endpoints REST Obrigatórios

Método	Endpoint	Descrição
POST	/auth/register	Cadastro com validação CPF+email, hash bcrypt, email confirmation
POST	/auth/login	Login → JWT access (15min) + refresh (7d) httpOnly cookie
POST	/auth/refresh	Rotate refresh token, invalida o anterior
GET	/market/prices	Snapshot preços (REST fallback ao WebSocket)
GET	/market/book/:ticker	Bid/Ask + OFI + spread em tempo real
POST	/orders	Criar ordem: market, limit, oco, short, scheduled
DELETE	/orders/:id	Cancelar ordem pendente

GET	/portfolio	Carteira + P&L + shorts abertos
GET	/transactions	Histórico paginado (cursor-based) com filtros
GET	/news	Proxy feed notícias (footstock-news-server)
POST	/ai/analyze	Proxy seguro Anthropic — requer auth + rate limit
POST	/payments/subscribe	Criar assinatura Mercado Pago
POST	/payments/webhook	Receber confirmações gateway (valida HMAC-SHA256)
GET	/admin/users	Lista usuários (role: admin+, paginado)
PATCH	/admin/users/:id	Atualizar role, plan, suspended

04

Autenticação e Segurança

4.1 Sistema Atual — NÃO usar em produção

■ Senhas em plaintext no bundle público

USERS_DB contém passwords em texto puro no JS enviado ao browser. O hash SHA-256 do admin é fixo no código-fonte. Ambos devem ser substituídos por bcrypt + PostgreSQL.

4.2 JWT RS256 (Target)

JAVASCRIPT

```
// Gerar par de chaves (uma vez, fora do código)
$ openssl genrsa -out private.pem 2048
$ openssl rsa -in private.pem -pubout -out public.pem

const jwt = require('jsonwebtoken');
const PRIV = require('fs').readFileSync('./private.pem');
const PUB = require('fs').readFileSync('./public.pem');

function signToken(user) {
  return jwt.sign(
    { sub: user.id, email: user.email, plan: user.plan, role: user.role },
    PRIV, { algorithm: 'RS256', expiresIn: '15m' }
  );
}

function verify(token) { return jwt.verify(token, PUB, { algorithms: ['RS256'] }); }
```

4.3 Proxy Seguro para Anthropic API

■ Chave exposta no frontend atual

footstock-gold.jsx chama <https://api.anthropic.com/v1/messages> sem x-api-key explícita (funciona via proxy claude.ai). Em deploy próprio, a chave DEVE estar apenas no servidor.

JAVASCRIPT

```
// backend/ai-proxy.js
app.post('/ai/analyze', authenticate, rateLimit({windowMs:3600000,max:10}), async (re
const resp = await fetch("https://api.anthropic.com/v1/messages", {
  method: "POST",
  headers: {
    "x-api-key": process.env.ANTHROPIC_API_KEY, // nunca no cliente
    "anthropic-version": "2023-06-01",
    "Content-Type": "application/json"
  },
  body: JSON.stringify(req.body)
});
res.json(await resp.json());
});
```

4.4 Rate Limiting

Endpoint	Limite	Janela	Ação
POST /auth/login	5 tentativas	15 minutos por IP	Evita quebra brute force
POST /auth/register	3 cadastros	1 hora por IP	Evita criação em massa
POST /orders	100 ordens	1 minuto por usuário	Evita flood de ordens
POST /ai/analyze	10 requests	1 hora por user	Controla custo de IA
POST /payments/webhook	Validar HMAC-SHA256		Evita spoofing de pagamento
Admin session	Timeout 30min	Inatividade	SESSION_TIMEOUT_MS implementado

05

Integrações Externas

5.1 Anthropic API — dois usos distintos

- **Assessor IA** — claude-sonnet-4-20250514 + tool web_search. Retorna JSON com resumo, sentimento, recomendação e notícias recentes.
- **Classificação de notícias** — Claude Haiku (mais barato) para extrair ticker e sentimento (-1 a +1) de manchetes RSS.

JSON

```
// Resposta esperada do Assessor (JSON puro sem markdown)
{ "resumo": "...", "pontos_positivos": [...], "pontos_negativos": [...],
  "sentimento_geral": "BULLISH|NEUTRO|BEARISH",
  "recomendacao": "COMPRAR|MANTER|VENDER", "nivel_risco": "BAIXO|MEDIO|ALTO",
  "noticias_recentes": [{ "titulo": "...", "sent": 0.8, "emoji": "■" }] }
```

5.2 Mercado Pago — Assinaturas Recorrentes

JAVASCRIPT

```
// npm install mercadopago
const mp = require('mercadopago');
mp.configure({ access_token: process.env.MP_ACCESS_TOKEN });

// Criar plano de assinatura
const plan = await mp.preapproval_plan.create({
  reason: "FootStock Craque",
  auto_recurring: { frequency: 1, frequency_type: 'months',
    transaction_amount: 19.90, currency_id: "BRL" }
});

// Webhook — POST /payments/webhook
// Validar HMAC-SHA256 com MP_WEBHOOK_SECRET antes de processar!
if (event.type === 'subscription_authorized_payment') {
  await db.updateUserPlan(userId, plan, nextBillingDate);
  await redis.del(`session:${userId}`); // forçar re-login com novo plano
}
```

5.3 News Server (footstock-news-server)

O app referencia `window.FOOTSTOCK_NEWS_API` || `"http://localhost:3001"`. O servidor deve:

- Expor **GET /api/news** → array de notícias com {ticker, sentimento, categoria, headline, ts}
- Buscar RSS (ESPN Brasil, GloboEsporte, Lance!) a cada 5 minutos
- Classificar com Claude Haiku para extrair ticker e sentimento (-1 a +1)
- Cache Redis 5min para não exceder rate limit da API
- Filtrar notícias com mais de 12h de idade (janela `NEWS_MAX_AGE` no cliente)

Sistema de Permissões (Admin)

6.1 Matriz ROLE_TABS — Acesso por Módulo

ROLE	dashboar d	motor	usuarios	financeir o	engaj.	moderac ao	noticias	patroc.
SUPERADM IN	✓	✓	✓	✓	✓	✓	✓	✓
ADMIN	✓	✓	✓	✓	✓	✓	✓	✓
MONITOR	✓	—	✓	—	✓	✓	—	—
EDITOR	✓	—	✓	—	—	✓	✓	—
MODERAD OR	—	—	—	—	—	✓	✓	—

6.2 Hierarquia de Roles e Ações (ROLE_USER_ACTIONS)

Cor	Ações Permitidas
#ffd700	Tudo: promote_admin, promote_monitor, promote_editor, promote_moderador, demote, suspend, unsuspend, reset, promote_lenda, prom
#e05555	Igual ao superadmin exceto promote_admin (não pode criar outro admin)
#f97316	Somente leitura — nenhuma ação de usuário
#C9A84C	suspend, unsuspend, reset, promote_lenda, promote_craque
#D4B466	Nenhuma ação de usuário — apenas conteúdo

6.3 Session Timeout (Admin)

JAVASCRIPT

```
// SESSION_TIMEOUT_MS = 30 * 60 * 1000 (30 minutos)
// Eventos que resetam o timer: mousemove, keydown, mousedown, touchstart, scroll
// Em produção: mover para backend – JWT expiration + refresh token rotation
// Redis: SETEX session:{userId} 1800 <session-data>
```

Configurações do Protótipo

7.1 IMPACT_MATRIX — Impacto de Notícias

Categoria	Impacto FV	Cap Spot	Exemplo
Financeira Crítica	±5.0%	±2.5%	Patrocínio R\$120M / bloqueio judicial
Esportiva Majoritária	±3.0%	±2.5%	Título Libertadores / eliminação
Mercado de Ativos	±2.0%	±2.5%	Venda de artilheiro €45M
Integridade / Saúde	±1.5%	±2.5%	Doping / fratura de capitão
Institucional	±1.0%	±2.5%	Eleição de presidente / aporte SAF
Esportiva Menor	±0.5%	±2.5%	Jogo-treino / promoção de jovem

7.2 AGENT_PROFILES — Liquidez Simulada por Cluster

Cluster	Ativos	prob/tick	Vol/ordem	Cooldown
A_TOP	URU3 POR4 TIM3 FOG3 GAL3	10%	0.002–0.008% float	4–8 ticks
A_MID	10 clubes médios Série A	5%	0.001–0.005% float	8–16 ticks
A_SMALL	5 clubes pequenos Série A	3%	0.0005–0.003% float	12–24 ticks
B_LIQUID	VOZ3 LEP4 COE3 IND4 LEI3...	2%	0.0003–0.002% float	16–32 ticks
B_ILLIQ	Demais Série B (13 ativos)	1%	0.0002–0.001% float	25–50 ticks

08

Deploy e Infraestrutura

8.1 Variáveis de Ambiente (.env.production)

```
# Banco de dados
DATABASE_URL=postgresql://user:pass@host:5432/footstock
REDIS_URL=redis://user:pass@host:6379

# Auth
JWT_PRIVATE_KEY_PATH=./private.pem
JWT_PUBLIC_KEY_PATH=./public.pem
REFRESH_TOKEN_SECRET=<256-bit-random>

# Anthropic
ANTHROPIC_API_KEY=sk-ant-...

# Mercado Pago
MP_ACCESS_TOKEN=APP_USR-...
MP_WEBHOOK_SECRET=<hmac-secret>
MP_PLAN_CRAQUE_ID=<mp-plan-id>
MP_PLAN_LENDA_ID=<mp-plan-id>

# App
PORT=3000
FRONTEND_URL=https://footstock.app
SESSION_TIMEOUT_MS=1800000
MARKET_TICK_MS=2000
SENTRY_DSN=https://...
```

BASH

8.2 Docker Compose (Desenvolvimento Local)

YAML

```
# docker-compose.yml
services:
  postgres:
    image: postgres:16-alpine
    environment: { POSTGRES_DB: footstock, POSTGRES_USER: dev, POSTGRES_PASSWORD: dev }
    ports: ["5432:5432"]

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass dev
    ports: ["6379:6379"]

  api:
    build: ./backend
    ports: ["3000:3000"]
    env_file: [.env.development]
    depends_on: [postgres, redis]

  frontend:
    build: ./frontend
    ports: ["5173:5173"]
    environment:
      VITE_API_URL: http://localhost:3000
      VITE_WS_URL: ws://localhost:3000
```

8.3 CI/CD Pipeline (GitHub Actions)

YAML

```
# .github/workflows/deploy.yml
on: { push: { branches: [main] } }
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16, env: { POSTGRES_DB: test ... } }
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test

  deploy:
    needs: test
    steps:
      - run: docker build -t footstock-api ./backend
      - run: docker push ghcr.io/footstock/api:${{ github.sha }}
      - run: render deploy --service-id ${ secrets.RENDER_SERVICE_ID }
```


Escalabilidade e Performance

9.1 Projeção de Carga

Métrica	Fase 1 (lançamento)	Fase 2 (6 meses)	Fase 3 (1 ano)
Usuários ativos/dia	500	5.000	50.000
Ordens/min (pico)	50	500	5.000
Conexões WebSocket	200	2.000	20.000
Ticks de preço/s	40/s (fixo — 40 ativos)		40/s
Writes BD/min	~500	~5.000	~50.000
API IA req/dia	~200	~2.000	~20.000
Custo Anthropic/mês	~\$20	~\$200	~\$2.000

9.2 Estratégia de Cache com Redis

REDIS

```
// Chaves Redis e TTLS
prices:{ticker}      HASH      TTL:2s      (motor publica via pub/sub)
book:{ticker}        HASH      TTL:2s      (bid/ask + OFI)
fv:{ticker}          STRING    TTL:10s     (Fair Value)
session:{user_id}    HASH      TTL:30m     (dados do usuário autenticado)
rate:login:{ip}      INCR      TTL:15m     (max 5 tentativas)
rate:ai:{user_id}    INCR      TTL:1h      (max 10 req/hora)
news:feed            LIST      TTL:5m      (cache do feed RSS)
circuit:{ticker}     STRING    TTL:5m      (circuit breaker ativo)
webauthn:challenge:{uid} STRING  TTL:60s   (WebAuthn nonce)

// Pub/Sub para broadcast de preços
PUBLISH market:prices <prices-json> // Motor → 2s
SUBSCRIBE market:prices                // API server → Socket.io.emit()
```

9.3 Consistência do Motor Distribuído

O maior desafio de escalar é o motor de mercado. Estratégias recomendadas:

- **Single-process leader** — apenas uma instância ativa. Leader election via Redis SETNX ou Redlock npm package.
- **Sticky sessions WS** — nginx ip_hash ou Socket.io Redis adapter para rotear cada cliente para a instância correta.
- **Estado no Redis, não em RAM** — prices[], fv[], imb[] armazenados no Redis. Restart da instância não perde estado.
- **Horizontal scaling do API** — servidores REST escalam livremente pois leem estado do Redis. Apenas o motor precisa ser single-leader.

Conformidade e LGPD

A Lei Geral de Proteção de Dados (Lei 13.709/2018) exige implementações que ainda não estão no protótipo:

Obrigação LGPD	Status	Ação Necessária
Consentimento explícito	✓ Checkbox na UI	Armazenar data/IP do consentimento no BD (tabela consents)
Direito de acesso	✗ Não implementado	GET /users/me/data → export JSON completo dos dados
Direito ao esquecimento	✗ Não implementado	DELETE /users/me → anonimização + remoção cascadeada
Portabilidade de dados	✗ Não implementado	GET /users/me/export → CSV/JSON para download
Log de acesso a dados	✗ Não implementado	Tabela data_access_logs (user, accessed_by, timestamp)
DPO identificado	✗ Não documentado	Nomear DPO e publicar e-mail de contato no app
Incidente de segurança	✗ Sem procedimento	SLA 72h para notificação à ANPD (anpd.gov.br)
Dados de menores	✓ Checkbox 18+	Validação server-side de CPF + data nascimento
Política de retenção	✗ Não definida	Definir: 5 anos financeiro, 90 dias logs, 30 dias sessões

★

Prioridade antes do lançamento público

Obrigatório: (1) Armazenar consentimento com timestamp e IP. (2) Implementar rota de exclusão de conta. (3) Definir política de retenção de dados. (4) Nomear e publicar contato do DPO.